Chapter 1

System Modelling and Residual-Force Formulation

Chapter **??** surveyed families that measure a residual, predict one, plan over one, or learn a corrective action with no force estimate at all. Those families are comparable as *controllers*, on the same vehicles, trajectories and formations. They are not required to compute the same internal quantity.

This chapter defines the residual acceleration a_{res} that *this* stack measures and logs. Two things follow. Strategies 1, 2 and 4, and any other method that consumes a force residual, then refer to one number rather than to several differently-defined ones. And a predicted residual can be checked against a measured one on the same flight, which is how a learned model is evaluated here.

The chapter also states the assumptions the formulation rests on, and where each one is expected to fail. Several are known to be imperfect, the summation in Section 1.4 most conspicuously, and stating them here is what allows the discussion chapter to interpret a negative result as evidence about an assumption rather than as an unexplained failure.

1.1 Notation and conventions

Vectors are expressed in a world frame that is *east-north-up*: the third basis vector e_3 points away from the ground and gravity acts along $-e_3$. This matches the convention used by the flight software and the motion-capture system, and is stated explicitly because the opposite convention is common in the quadrotor literature and the sign of the gravity term differs between them.

For vehicle i: $p_i \in \mathbb{R}^3$ is position, $v_i \in \mathbb{R}^3$ velocity, $R_i \in SO(3)$ the rotation from body to world, and $\omega_i \in \mathbb{R}^3$ the body angular velocity. The hat map $\hat{\cdot} : \mathbb{R}^3 \to \mathfrak{so}(3)$ is defined by $\hat{a}b = a \times b$. The vehicles are homogeneous, of mass m and diagonal inertia J. Rotor k spins at ϖ_k and produces thrust $k_{t,k}\varpi_k^2$ along the body z-axis.

1.2 Rigid-body model with a residual wrench

Each vehicle is modelled as a rigid body driven by a collective thrust f_i along its body z-axis and a body torque τ_i:

$$\dot{p}_i = v_i, \qquad m\,\dot{v}_i = -mge_3 + f_i R_i e_3 + f_{\text{res},i}, \tag{1.1}$$

$$\dot{R}_i = R_i \hat{\omega}_i, \qquad J\dot{\omega}_i = -\omega_i \times J\omega_i + \tau_i + \tau_{\text{res},i}. \tag{1.2}$$

The terms $f_{\text{res},i}$ and $\tau_{\text{res},i}$ form the *residual wrench*: by construction they collect every force and moment that (1.1)–(1.2) otherwise omits. This includes blade flapping, rotor drag, ground effect and errors in m, J and k_t, as well as the inter-vehicle aerodynamic interaction that is the subject of this thesis.

That the residual is a catch-all rather than a physical quantity has two consequences that matter for the experiments. First, a residual measured in isolated free flight is not zero; it is whatever the nominal model gets wrong about a single vehicle. Second, the interaction can only be isolated by *differencing*, comparing the residual measured in formation against the residual measured alone under the same trajectory. The coplanar control scenarios in Chapter **??** exist for exactly this purpose.

Scope: force, not moment This thesis compensates the *force* residual f_{res}. The moment residual τ_{res} is measured and logged, but no strategy compensates it separately: the attitude loop rejects it as a disturbance in the usual way. This is a scoping decision, not a claim that τ_{res} is negligible.

The decision has company. Neural-Swarm compensates the vertical interaction force [7], and the hardware controller of Hsieh et al. [4] likewise commands thrust and angular rates, leaving residual torque to the rate loop. Kiran et al. [5] and Gielis et al. [2] do measure both force and moment, but on a rig rather than as a flight residual.

We therefore log τ_{res} and do not cancel it as a separate term. The cost is that interaction torque is left to the attitude loop. The benefit is a single compensation channel that fits the existing allocator and the onboard budget. The discussion chapter returns to what leaving it uncompensated may cost.

1.3 Measuring the residual

The residual can be recovered without any model of the interaction, by comparing what the vehicle actually accelerated with what the nominal model says it should have accelerated given the actuation actually applied. Rearranging (1.1),

$$\frac{f_{\text{res},i}}{m} = \underbrace{\dot{v}_i}_{a_{\text{meas}}} - \underbrace{\left(\frac{f_i}{m} R_i e_3 - g e_3\right)}_{a_{\text{model}}}. \tag{1.3}$$

Both terms are available onboard. The measured acceleration comes from the accelerometer, which senses specific force in the body frame and therefore excludes gravity by construction; rotating it into the world frame and subtracting ge_3 gives a_{meas}. The model acceleration requires the applied thrust, which is reconstructed from measured rotor speeds as $f_i = \sum_{k=1}^{4} k_{t,k} \varpi_k^2$.

The implementation evaluates (1.3) at every control step and logs it as a_{res}, in every controller mode, including modes that make no use of it. This is deliberate. The training data for every predictive strategy is collected under the *uncompensated* geometric controller, so the measurement must exist when no INDI term is active; and the comparison of a predicted residual against a measured one, which is how a learned model is evaluated, requires both quantities on the same flight.

1.3.1 What this measurement requires, and how it fails

Three properties of (1.3) shape the experimental design.

It requires rotor-speed sensing. Without ϖ_k the applied thrust is unknown and a_{model} cannot be formed. This is the hardware prerequisite identified in Section **??**, and the same one that Cobo-Briesewitz et al. [1] remove by learning the residual instead of computing it. In the implementation, the absence of a rotor-speed source causes a_{res} to read *identically zero*. That failure is silent and is indistinguishable, in a log file, from a flight in which no interaction occurred. Verifying that $a_{\text{res}} \neq 0$ is therefore the first check of every experimental session (Chapter **??**).

It is delayed. a_{meas} is differentiated from noisy inertial measurement and must be filtered; the reconstructed thrust inherits the rotor-speed sensing delay. The estimate therefore exists only after the disturbance has begun to act. No amount of tuning removes this: it is the structural property that motivates prediction.

It absorbs model error. Any error in m or k_t appears in a_{model} and is attributed to f_{res}. A miscalibrated thrust constant produces a residual that looks like a constant interaction force. This is why the airframe constants are identified once and frozen, and why the null scenarios matter: a residual that persists when no neighbour is present is model error, not interaction.

1.3.2 Sign convention in the control law

The compensation enters with a sign that is easy to get wrong and whose error is not obvious in flight. From (1.1), holding a desired acceleration a_{des} requires

$$f_i R_i e_3 \;=\; m\, a_{\text{des}} + mge_3 - f_{\text{res},i}, \tag{1.4}$$

so the desired-acceleration vector passed to the thrust-and-attitude allocation carries $-f_{\text{res},i}/m$. The residual is *subtracted*, whether it was measured (Section 1.3) or predicted (Section 1.4).

Adding it instead does not merely fail to compensate: it doubles the disturbance, because the controller then commands a thrust error equal and opposite to the correction it should have made. The resulting behaviour is a stable, well-controlled vehicle that tracks slightly worse in formation, which is what an uncompensated vehicle also looks like. In single-vehicle flight the error is invisible altogether, since $f_{\text{res}} \approx 0$ with no neighbour present. This is recorded here because it is a property of the formulation rather than a coding detail, and because it establishes a specific prediction that the two-robot results chapter can check: with the sign correct, position-INDI compensation must *reduce* the formation-induced displacement, and with it inverted the displacement must be approximately twice the uncompensated value.

1.4 Predicting the residual from relative state

The interaction force on vehicle i is a function of the configuration of the team relative to it,

$$f_{\text{res},i} \;\approx\; \mathcal{F}(\{\, p_j - p_i,\; v_j - v_i \,\}_{j\neq i}), \tag{1.5}$$

and (1.5) is what a predictive strategy learns. Two structural properties are required of any parameterisation of $\mathcal{F}$, and both follow from the physics rather than from convenience. The value must not depend on the order in which neighbours are enumerated, and it must be defined for a varying number of them, since a vehicle in a three-robot team has two neighbours and one in a pair has one.

1.4.1 Deep-sets parameterisation

The parameterisation used here satisfies both by construction:

$$\hat{a}_{\mathrm{res},i} \;=\; \rho\left(\sum_{j\neq i}\phi(p_j - p_i,\; v_j - v_i)\right), \tag{1.6}$$

where ϕ is applied to each neighbour independently and ρ maps the summed embedding to a residual acceleration in the world frame. Summation is permutation-invariant and accepts any number of terms, so one parameter set serves teams of different sizes. The output is an acceleration rather than a force so that it is directly comparable with the measured a_{res} of (1.3); the two quantities are in the same units, in the same frame, at the same point in the control loop.

What is borrowed and what is ours. The *family* is the permutation-invariant deep set used by Neural-Swarm and Neural-Swarm2 [7, 6]. The layer widths and the resulting 987 parameters are ours, sized for the STM32 control period. This is not the network of Cobo-Briesewitz et al. [1], and it is not a copy of Neural-Swarm2's layers. Spectral normalisation is not used here. An architecture ablation would be a separate study and is out of scope unless time remains after the comparison.

The concrete network is small: ϕ maps $6 \to 16 \to 16 \to 8$ and ρ maps $8 \to 16 \to 16 \to 3$, with rectified-linear activations throughout except at the output, which is linear because an interaction force may point either way. This is 987 parameters, evaluated onboard within the control period. The sizing is discussed in Chapter **??**; what matters here is that (1.6) is the model, and the layer widths are an implementation choice within it.

These equations are the code. Equation (1.6), the layer widths above, the distance guards of Section 1.4.3 and the output bound are the values in the residual-network source file,[1] not an idealisation of them, and were read from it rather than reconstructed. The same applies to (1.3) and the sign convention of Section 1.3.2. If the implementation changes, this chapter changes with it; a modelling chapter that has drifted from the flight code describes a system that was never flown.

1.4.2 The superposition assumption

Equation (1.6) assumes the aggregate interaction is a sum of per-neighbour contributions in the latent space of ϕ. This is more general than summing forces directly, ρ is free to be nonlinear in the sum, but it is still an assumption, and the literature states plainly that it does not hold exactly. Shi et al. [7] report that with more than two vehicles the aerodynamic effect is not a simple superposition of each pair, and Gielis et al. [2] motivate a learned aggregation model, measured on a load stand, precisely on the grounds that a single-vehicle downwash model is unlikely to suffice for predicting aggregate forces.

The assumption is retained here for two reasons. It is what makes one trained model applicable to teams of two and three without retraining, which the comparison requires. And a controller does not need an exact model: it needs one whose residual after compensation is smaller than the residual before it. The relevant question is therefore not whether (1.6) is exact, which it is not, but how large the resulting error is on a deployed model, which is what the multi-robot results chapter measures.

[1] `firmware_app/src/residual_nn.rs` for the network; `firmware_app/src/lib.rs` for the residual computation and the desired-acceleration assembly.

1.4.3 Input conditioning

Two guards are applied to the inputs of ϕ, and both are part of the model rather than defensive programming, because they change the function being learned.

A neighbour farther than 2 m is excluded from the sum. Beyond that distance the interaction is below the noise floor of the measurement, and including such neighbours would ask the network to learn that distant vehicles do nothing, spending capacity on a region where the answer is known.

A separation closer than 4 cm is clamped to that value, preserving direction. Below it the model would extrapolate beyond anything the training data can contain, since the vehicles are 9 cm across and such a configuration is a collision. A learned function is least trustworthy exactly where the physics is strongest, and the clamp bounds the extrapolation rather than trusting it.

The output is bounded as well, at $8\,\mathrm{m/s^2}$, comfortably above the interaction accelerations expected between vehicles of this size, and far below what an ill-conditioned network can emit. This is not a hypothetical failure mode. In simulation, Gu et al. [3] report a radial-basis-function estimator crashing the vehicle under periodic wind, and an extreme learning machine becoming unstable on landing under ground effect; their echo state network and Gaussian process did not. The authors say the failure *may be attributed* to the random initialisation of the input-to-hidden weights, which *"can produce excessively large prediction values"*. Their vehicle is a single quadrotor and their disturbances are wind and ground effect, so this is not evidence about inter-vehicle interaction, but it is evidence that an unbounded learned term can destabilise a controller. A saturating output converts that failure into a bounded and, because saturation is logged, a visible one.

The training data must pass through the same guards as the deployed model. A model fitted to unguarded inputs is fitted to a different distribution than the one it will meet, and the discrepancy is invisible until predicted and measured residuals are compared in flight.

1.4.4 Normalisation

Input scaling is folded into the first layer of ϕ at export rather than applied onboard. For weights W_1 and bias b_1 trained on inputs normalised by mean μ and standard deviation σ,

$$W_1 \frac{x-\mu}{\sigma} + b_1 \;=\; \underbrace{(W_1 \oslash \sigma)}_{W_1'}\, x + \underbrace{\big(b_1 - (W_1 \oslash \sigma)\,\mu\big)}_{b_1'}, \tag{1.7}$$

with $\oslash$ denoting column-wise division. The two forms are arithmetically identical, so the deployed network computes the same function as the trained one while the flight software carries no normalisation constants that could fall out of step with the model.

1.5 Assumptions and where they fail

Table 1.1 collects the assumptions this formulation makes. Each is stated with the condition under which it is expected to break, so that an experimental result can be attributed to a specific assumption rather than to the model as a whole.

Two entries deserve emphasis because they are not merely caveats but hypotheses the experiments test. The superposition assumption is known to be imperfect, and the experimental question is the size of the penalty rather than its existence. And the training-distribution assumption is exactly what separates the predictive family from the reactive

Table 1.1: Assumptions of the residual formulation and their expected failure modes.

Assumption	Where it fails	Consequence
Rigid body; no aeroelastic deformation	Not expected to fail at this scale	—
m, J, k_t known and constant	Battery discharge; rotor wear; miscalibration	Model error appears as a constant residual; bounded by the null scenarios
Residual is dominated by inter-vehicle interaction	Near the ground; near walls	Flight volume is bounded away from both (Chapter **??**)
Neighbour relative states are known accurately	Motion-capture dropout; latency	Prediction degrades; measurement is unaffected, which is itself a comparison result
Aggregate interaction superposes in latent space	Three or more vehicles, overlapping wakes	**Known to be imperfect** [7, 2]; magnitude measured in the multi-robot results chapter
Interaction lies within the training distribution	Unseen separations, faster relative motion	Prediction degrades while measurement does not; this is the crossover of **RQ2**
Rotor-speed measurement available	No RPM source fitted	a_{res} reads identically zero — silent, and checked every session

one: a measurement has no training distribution to leave. Cobo-Briesewitz et al. [1] observe precisely this effect for their purely learned method in the no-payload case, attributing a drop in performance on the circular trajectory to its being out of distribution relative to the training data.

1.6 Summary

The residual of (1.1)–(1.2) is what this stack measures and logs. Strategies 1, 2 and 4 consume it. The residual can be recovered online from (1.3), which needs no model of the interaction but requires rotor-speed sensing and is available only after the disturbance acts; or predicted from (1.6), which acts ahead of the disturbance but is valid only within its training distribution and assumes an aggregation that is known to be approximate. Both are expressed as accelerations in the world frame and enter the control law at the same point with the same sign, (1.4).

That common definition is what makes the comparison in the two-robot results chapter a comparison rather than a collection of separately-tuned controllers. Chapter **??** describes the strategies built on it, and the point at which each injects its knowledge of the interaction.

Bibliography

[1] Eckart Cobo-Briesewitz, Khaled Wahba, and Wolfgang Hönig. Learned incremental nonlinear dynamic inversion for quadrotors with and without slung payloads. arXiv:2503.09441, 2025. Published in Proceedings of the 8th Annual Conference on Learning for Dynamics and Control (L4DC), PMLR.

[2] Jennifer Gielis, Ajay Shankar, Ryan Kortvelesy, and Amanda Prorok. Modeling aggregate downwash forces for dense multirotor flight. arXiv:2312.03488, 2023.

[3] Weibin Gu, Jiance Zhao, and Alessandro Rizzo. Learning uncertainties online for quadrotor flight control: A comparative study. *Journal of Intelligent & Robotic Systems*, 2025. Abstract read via OpenAlex; full text not held (paywalled).

[4] Pei-An Hsieh, Fengjun Yang, Nikolai Matni, and M. Ani Hsieh. Flatness-preserving residual learning for real-time tight quadrotor formation flight. arXiv:2607.12275, 2026.

[5] Anoop Kiran, Nora Ayanian, and Kenneth Breuer. Influence of static and dynamic downwash interactions on multi-quadrotor systems. arXiv:2507.09463, 2025.

[6] Guanya Shi, Wolfgang Hönig, Xichen Shi, Yisong Yue, and Soon-Jo Chung. Neural-swarm2: Planning and control of heterogeneous multirotor swarms using learned interactions. arXiv:2012.05457, 2020. Published in IEEE Transactions on Robotics.

[7] Guanya Shi, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung. Neural-swarm: Decentralized close-proximity multirotor control using learned interactions. arXiv:2003.02992, 2020.